Uploading Your Project to GitHub

Intel Image Classification — ML + ORB Project

1. Create a GitHub Repository

1. Go to **github.com** and log in.

2. Click the **+** icon (top right) → **New repository**.

3. Name it, e.g. **intel-image-classification-ml**.

4. Choose **Public** (required by the assignment).

5. Do **NOT** tick "Add a README" if you already have one locally — otherwise leave it unchecked and add your own.

6. Click **Create repository**. Keep this page open — GitHub shows you the exact commands for the next steps.

2. Prepare Your Local Project Folder

Your folder should look like this before uploading:

```
intel-image-classification-ml/
■■■ notebook/
■ ■■■ intel-image-classification.ipynb
■■■ app.py
■■■ best_model.pkl
■■■ classes.json
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

3. Ignore Your Environment — .gitignore

Your **venv** folder (virtual environment) must NEVER be pushed to GitHub. It's large, machine-specific, and fully rebuildable from **requirements.txt**. Create a file named exactly **.gitignore** in your project root with this content:

```
# Virtual environment
venv/
env/
.venv/

# Python cache
__pycache__/
*.pyc

# Jupyter
.ipynb_checkpoints/

# OS files
.DS_Store
Thumbs.db

# Streamlit
.streamlit/
```

Note: keep **best_model.pkl** tracked (do NOT ignore it) — your reviewer needs it to run the app without retraining. Only the environment/cache folders should be ignored.

4. Initialize Git and Push

Open a terminal inside your project folder and run:

```
git init
git add .
git commit -m "Initial commit: ML pipeline + Streamlit app"
git branch -M main
git remote add origin https://github.com//intel-image-classification-ml.git
git push -u origin main
```

Replace <your-username> with your actual GitHub username. If prompted for a password, GitHub requires a Personal Access Token (not your account password) — generate one under GitHub → Settings → Developer settings → Personal access tokens.

5. Verify

- Refresh your GitHub repo page — you should see all files EXCEPT the venv/cache folders.
- Confirm README.md renders nicely on the repo's main page.
- Confirm best_model.pkl and classes.json are present (needed for the app to run for anyone who clones the repo).

6. Making Future Updates

Whenever you change files later:

```
git add .
git commit -m "Describe your change here"
git push
```